Part1. Clase globale

G	N	Z	B	S	I	Rezultat ast
G1	N1	Z1	B1	S1	I6	O1 == 0.0
G2	N2	Z2	B2	S2	I1	O1 = 0.0
G3	N3	Z1	B3	S3	I2	O2 >0
G4	N4	Z3	B4	S4	I3	O2 = 10.0
G5	N3	Z4	B5	S5	I4	O2>0
G6	N1	Z5	B6	S6	I5	O1 =0.0
G7	N3	Z2	B3	S4	I3	O2 >0

G1 = N1 x Z1 x B1 x S1 x I6 = O1
G2 = N2 x Z2 x B2 x S2 x I1 = O1
G3 = N3 x Z1 x B3 x S3 X I2 = O2
G4 = N4 x Z3 x B4 x S4 X I3 = O2
G5 = N3 x Z4 x B5 x S5 X I4 = O2
G6 = N1 x Z5 x B6 x S6 X I5 = O2
G7 = N3 x Z2 x B3 x S4 x I3 = O2

Clasa glo	cvss_score	zona_retea	Importanta _business	severitate	Istoric_ inci	Rezultat ast
G1	<0 ⇔ -3	External 1.5	critica1.5	Critical 1.4	None	0.0
G2	0.0	DMZ	mare	high	[]	0.0
G3	4	internal=1.0	Medie 1.1	Medium 1.1	breach+1.5	6.34
G4	15~min(10)	‘internal’ = 1.(	‘medie’ 1.1	=medium =	malware+0.	min(12.6, 1(
G5	15~min(10)	‘Cloud’ = 1.0	‘Imensa’ 1.1	‘Supermare’	Breach+1.5, ıre+0.5	min(13, 10)
G6	-7	None	None	None	0	0.0
G7	3	DMZ = 1.3	Medie 1.1	Low 1.1	Malware +0	5.2

La G4, fiindca luam min(G4, 10), nu ne putem da seama daca I3(malware, +0.5) este folosit sau nu, asa ca mai introducem un test.

Part2. Metoda grafului cauza efect
Tabele decizie

Matrice Efect 1 - critic

Cauze	V1	V2	V3	V4	V5	V6	V7
C1 >=9	1	1	1	1	0	0	0
C2 critical	1	0	0	0	1	1	1
C3 >= 7	0	0	0	0	1	0	0
C4 high	0	1	0	0	0	0	0
C5 >= 4	0	0	0	0	0	1	0
C6 mediur	0	0	1	0	0	0	0
C7 < 4	0	0	0	0	0	0	0
C8 low	0	0	0	1	0	0	1
Efect 1 crit	1	1	1	1	1	1	1

Matrice Efect 2 - ridicat - risk_score >= 7.00 or sev = ‘high’

Cauze	V8	V9	V10	V11	V12
C1 >=9	0	0	0	0	0
C2 critical	0	0	0	0	0
C3 >= 7	1	1	1	0	0
C4 high	1	0	0	1	1
C5 >= 4	0	0	0	1	0
C6 medium	0	1	0	0	0
C7 < 4	0	0	0	0	1
C8 low	0	0	1	0	0
Efect 2 ridicat	1	1	1	1	1

Matricea 3 - mediu - risk_score >= 4 or sev = ‘medium’

Cauze	V13	V14	V15
C1 >=9	0	0	0
C2 critical	0	0	0
C3 >= 7	0	0	0
C4 high	0	0	0
C5 >= 4	1	1	0
C6 medium	1	0	1
C7 < 4	0	0	1
C8 low	0	1	0
Efect 1 critic	1	1	1

Matricea 4 - low - risk_score <4 or sev = ‘low’

Cauze	V1
C1 >=9	0
C2 critical	0
C3 >= 7	0
C4 high	0
C5 >= 4	0
C6 medium	0
C7 < 4	1
C8 low	1
Efect 1 critic	1

Part3. CDG - graful de control al fluxului

```
class RiskScorer():
    M_RETEA = {
        'external': 1.5,    # expus in internet — risc maxim
```

```
        'dmz':      1.3,   # zona demilitarizata — risc mediu-mare
        'internal': 1.0,   # retea interna izolata — risc de baza
    }
     M_BUSINESS = {
        'critica': 1.5,   # sisteme critice (ERP, baze de date productie)
        'mare':    1.3,   # sisteme importante (servere aplicatii)
        'medie':   1.1,   # sisteme moderate (workstations)
        'scazuta': 1.0,   # sisteme necritice (imprimante, IoT minor)
    }
       M_AMENINTARE = {
        'critical': 1.4,   # exploatata activ in atacuri reale
        'high':     1.2,   # exploit public disponibil
        'medium':   1.1,   # exploatarea este teoretic posibila        'low':
1.0,   # fara exploit cunoscut
    }
    @staticmethod
    def calculeaza_risk_score(
        cvss_score: float,
        zona_retea: str,
        importanta_business: str,
        severitate: str,
        istoric_incidente = None
    ) -> float:
    1    if cvss_score < 0.0:
    2  cvss_score = 0.0
    3   m_retea = RiskScorer.M_RETEA.get(zona_retea.lower()
                               if zona_retea else 'internal', 1.0)
        m_business = RiskScorer.M_BUSINESS.get(importanta_business.lower()
                                    if importanta_business else 'medie', 1.1)
        m_amenintare = RiskScorer.M_AMENINTARE.get(severitate.lower()
                                         if severitate else 'low', 1.0)
    4  raw_score = cvss_score * m_retea * m_business * m_amenintare
       factor_penalizare = 0.0
     5  if istoric_incidente:
     6       for incident in istoric_incidente:
     7          if incident == 'breach':
     8             factor_penalizare += 1.5
     9          elif incident == 'malware':
     10               factor_penalizare += 0.5
     11   raw_score += factor_penalizare
        normalized = min((raw_score / 31.5) * 10, 10.0)
        return round(normalized, 2)
```

```python
@staticmethod
    def genereaza_recomandare(
        risk_score: float,
        severitate: str,
        remediation: str = None
    ) -> dict:
        sev = (severitate or 'low').lower()
        if risk_score >= 9.0 or sev == 'critical':
            return {
                "urgenta":    "CRITIC — Actiune imediata necesara",
                "termen":     "< 24 ore",
                "prioritate": 1,
                "actiune":    remediation or
                              "Patch imediat sau izolati sistemul de reteaua locala. "
                              "Contactati echipa de securitate.",
                "culoare":    "#ef4444",  # rosu
            }
        elif risk_score >= 7.0 or sev == 'high':
            return {
                "urgenta":    "RIDICAT — Remediere urgenta",
                "termen":     "< 7 zile",
                "prioritate": 2,
                "actiune":    remediation or
                              "Aplicati patch-ul disponibil sau implementati masuri
temporare de protectie ",
                "culoare":    "#f59e0b",  # portocaliu
            }
        elif risk_score >= 4.0 or sev == 'medium':
            return {
                "urgenta":    "MEDIU — Planificati remedierea",
                "termen":     "< 30 zile",
                "prioritate": 3,
                "actiune":    remediation or
                              "Includeti in planul de patching lunar. "
                              "Monitorizati pentru semne de exploatare.",
                "culoare":    "#3b82f6",  # albastru
            }
        else:
            return {
```

```
        "urgenta":    "SCAZUT — Remediere planificata",
        "termen":     "< 90 zile",
        "prioritate": 4,
        "actiune":    remediation or
                      "Includeti in ciclul normal de patch management.",
        "culoare":    "#22c55e",  # verde
    }
```